

Mastering Advanced Java & DSA - Jan '26



Day – 2 [Loops]

- Scenario Based Problem Solving on Loops
- Problem Solving on Nested loops.
- Advance problems on Loops
- Constructors,
- this super keyword in Java
- Instance initialized block, local variables vs instance variables



1. Scenario-Based Problem Solving on Loops

- Loops are used when a block of code needs to be executed **repeatedly** based on a condition.
- Scenario-based loop problems help students apply logic to **real-world situations** such as counting, summing, and reversing numbers.

Example Code – Sum of N Numbers

```
class SumNumbers {  
    public static void main(String[] args) {  
        int n = 5;  
        int sum = 0;  
  
        for (int i = 1; i <= n; i++) {  
            sum = sum + i;  
        }  
        System.out.println("Sum = " + sum);  
    }  
}
```

Explanation

- **n** represents how many numbers to add
- Loop runs from 1 to n
- **sum** accumulates the total
- Final sum is printed

2. Problem Solving on Nested Loops

Nested loops mean **one loop inside another loop**.

They are mainly used for **pattern printing, matrix operations, and table generation**.

Example Code – Right Angle Triangle Pattern

```
class Pattern {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Explanation

- Outer loop controls **rows**
- Inner loop controls **columns**
- Inner loop runs based on outer loop value
- Creates increasing star pattern

3. Advanced Problems on Loops

- Advanced loop problems involve **multiple conditions, break statements, and mathematical logic.**
- They improve **problem-solving and algorithmic thinking.**

Example Code – Prime Number Check

```
class Prime {  
  
    public static void main(String[] args) {  
  
        int num = 7;  
  
        boolean prime = true;  
  
        for (int i = 2; i <= num / 2; i++) {  
            if (num % i == 0) {
```

```
                prime = false;  
                break;}}  
  
        if (prime && num > 1)  
            System.out.println("Prime  
Number");  
  
        else  
            System.out.println("Not a  
Prime Number");}}}
```

Explanation

- Loop checks divisibility
- If divisible, number is not prime
- `break` stops loop early
- Final result printed using if-else

4. Constructors in Java

- A constructor is a **special method** used to **initialize objects**.
- It is called **automatically** when an object is created and has the same name as the class.

Example Code – Parameterized Constructor

```
class Student {  
    String name;  
  
    int age;  
  
    Student(String n, int a) {  
        name = n;  
  
        age = a;  
    }  
}
```

```
public static void main(String[]  
args) {  
  
    Student s = new  
Student("Pattu", 20);  
  
    System.out.println(s.name);  
  
    System.out.println(s.age);  
    }  
}
```

Explanation

- Constructor initializes instance variables
- Object creation calls constructor
- Values passed during object creation

5. **this** Keyword

- The **this** keyword refers to the **current object**.
- It is mainly used to differentiate **instance variables from local variables**.

Example Code

```
class Demo {  
  
    int x;  
  
    Demo(int x) {  
  
        this.x = x;  
  
    }  
  
    public static void main(String[] args) {  
  
        Demo d = new Demo(10);  
  
        System.out.println(d.x);  
  
    }  
  
}
```

Explanation

- **this.x** refers to instance variable
- **x** refers to constructor parameter
- Avoids naming conflict

6. **super** Keyword

The **super** keyword refers to the **parent class object**.

It is used to access **parent class variables, methods, and constructors**.

Example Code

```
class Parent {  
  
    int value = 100;  
  
}
```

```
class Child extends Parent {  
  
    int value = 50;  
  
    void show() {  
  
        System.out.println(value);  
  
        System.out.println(super.value);  
  
    }
```

```
public static void main(String[] args) {  
    Child c = new Child();  
    c.show();  
}  
}
```

Explanation

- value → child class variable
- super.value → parent class variable
- Resolves variable hiding

7. Instance Initialization Block (IIB)

An Instance Initialization Block is used to **initialize instance variables**.
It runs **before the constructor** every time an object is created.

Example Code

```
class IIBDemo {  
  
    int x;  
  
    {  
  
        x = 10;  
  
        System.out.println("Instance Block");  
  
    }  
  
    IIBDemo() {  
  
        System.out.println("Constructor"); }  
}
```

```
public static void main(String[] args) {  
    IIBDemo obj = new IIBDemo();  
    System.out.println(obj.x);  
}  
}
```

Explanation

- Instance block executes first
- Constructor executes next
- Used for common initialization

8. Local Variables vs Instance Variables

Local variables are declared **inside methods**,
Instance variables are declared **inside class but outside methods**.

Example Code

```
class VariableDiff {  
    int a = 10; // instance variable  
  
    void show() {  
        int a = 20; // local variable  
        System.out.println(a);  
        System.out.println(this.a);  
    }  
  
    public static void main(String[] args) {  
        VariableDiff v = new VariableDiff();  
        v.show();  
    }  
}
```


Explanation

- Local variable has higher priority
- `this.a` accesses instance variable
- Helps understand variable scope

Recommendations

1. Practice More Problems → Solve DSA coding challenges daily on platform CodeKatta.
2. Hands-on with Java Features → Implement custom collections, thread-safe programs, and practice design patterns.
3. Mini Projects → Build a Student Management System, Online Library, or Task Scheduler to apply Java + DSA together.
4. System Design Practice → Try designing Parking Lot, E-commerce Cart, or Chat Application using OOP + SOLID principles.
5. Deep Dive into DSA → Focus on problem patterns: Sliding Window, Binary Search on Answer, Graph Traversals, DP on Trees.
6. Competitive Programming → Set daily/weekly goals to solve problems under time limits to mimic interview pressure.

1. Code with Functional Style → Use Java 8+ features (Streams, Lambdas, Optional) in all projects for clean, modern coding.
2. Revise Complexity Analysis → Always analyze Big-O for your code; optimize where possible with better data structures.
3. Mock Assessments → Attempt coding tests (similar to HackerRank/CodeSignal) after each module to track progress.
4. Prepare for Interviews → Practice common interview patterns: Linked List cycle detection, DP Knapsack, Graph shortest path, and LLD problems.

Thank You!